Problem A. One Goal

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

There are n cities and $n-1$ roads in the Republic of Never. The cities are conveniently numbered from 1 to n. Every road connects two cities and can be traversed in both directions. It is possible to move between any two cities using the roads. The distance $d(u, v)$ is defined as the smallest number of roads one needs to use to move from city u to city v.

There are k friends who are looking to meet. The i-th friend lives in city a_i. For the meeting, the friends will choose city v such that $\sum_{i=1}^{k} d(v, a_i)$ is minimum. If there are several such cities, they will choose the one with the smallest number among them.

Unfortunately, you know just the number of friends but nothing about the cities where they live. Every friend might live in any of the n cities; hence, there are n^k options overall. You would like to find the sum of the numbers of cities the friends will choose in all the n^k options. Output this sum modulo 998 244 353.

Input

The first line of the input contains two integers n and k ($2 \le n, k \le 30\,000$), denoting the number of cities and the number of friends.

Each of the following $n-1$ lines contains two integers u_i and v_i ($1 \le u_i, v_i \le n$; $u_i \ne v_i$), denoting the numbers of the cities connected by the i-th road.

It is guaranteed that it is possible to move between any two cities using the roads.

Output

Display the sum of the numbers of cities the friends will choose in all the n^k options, modulo 998 244 353.

Examples

standard input	standard output
3 2 1 2 1 3	12
5 3 2 4 5 3 1 3 3 2	357

Note

In the first example, with three cities and two friends, there are $3^2 = 9$ options to consider. In three of these options, the friends are in the same city and they will clearly choose this city for the meeting. In the other six options, the friends are in different cities and they will choose city 1. The total is $(1 + 2 + 3) + 6 \cdot 1 = 12$.

Problem B. Two Teams

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

After five hours of intense battle, the final contest of XCPC is complete! Even though the standings were frozen during the last hour, it is obvious that there are just two contenders for the trophy, namely team A and team B. All other teams are ignored in this description from now on.

In XCPC, teams are ranked according to the most problems solved. Teams who solve the same number of problems are ranked by the least total penalty time. In case of a tie, team A is ranked above team B.

The number of problems solved and the penalty of both teams at the 4-hour mark are publicly known. You are responsible for showing the final standings at the closing ceremony, and you have been privately given a list of integers for each team. Each element of the list stands for a problem solved during the last hour, with its value denoting the amount of penalty the team will get for solving this problem.

At the closing ceremony, the standings will be unfrozen by revealing who solved which problems during the last hour, one by one. The following sequence of actions will be repeated until the standings are final:

1. Team X with the lowest rank and at least one more unrevealed solved problem is chosen.
2. It is revealed that team X actually solved one of its unrevealed solved problems. In the standings, the number of problems solved by team X increases by 1, the total penalty of team X increases by the corresponding amount, and the ranking of the teams is recalculated.

You have control over which problem will be chosen for revealing in step 2 at every moment. Every time teams A and B exchange their ranks, spectators will be excited. Find the maximum number of times you can make this happen.

Input

The input consists of two blocks of two lines each: a block for team A and a block for team B.

The first line of the i-th block contains two integers s_i and t_i ($1 \le s_i \le 100$; $1 \le t_i \le 30\,000$), denoting the number of problems solved and the penalty time received by the team during the first four hours.

The second line of the i-th block contains an integer n_i ($0 \le n_i \le 100$), denoting the number of problems solved by the team during the last hour, followed by n_i integers $a_{i,1}, a_{i,2}, \ldots, a_{i,n_i}$ ($241 \le a_{i,j} \le 1000$), denoting penalty times the team will receive for solving each of these problems.

Output

Display the maximum number of times teams A and B can swap positions in the standings.

Example

standard input	standard output
4 270 3 458 245 386 6 968 1 430	3

Note

In the example test case, initially team B is at the top of the standings. First, team A is chosen twice. It is optimal to choose problems with penalties 245 and 386, in any order; then team A gets ahead of team B, with 6 problems and penalty 901. After that team B gets ahead of team A, with 7 problems and penalty 1398. Finally, team A gets ahead of team B again, with 7 problems and penalty 1359.

Problem C. Three Indices

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

A string t is called a *smooth transformation* of a string w if there exist an integer $m \geq 1$ and strings $w_0, w_1, \ldots, w_m$ such that:

- $w_0 = w$, and $|w_i| = |w|$ when $0 < i \leq m$;
- w_i differs from w_{i-1} in at most one position when $0 < i \leq m$;
- $t = w_0 w_1 \ldots w_m$.

You are given a string $s = s_1 s_2 \ldots s_{|s|}$. Find the number of triplets of indices (i, j, k) such that $1 \leq i < j < k \leq |s|$ and $s_{i..k} = s_i s_{i+1} \ldots s_k$ is a smooth transformation of $s_{i..j} = s_i s_{i+1} \ldots s_j$.

Input

The only line of the input contains the string s ($4 \leq |s| \leq 10^5$) consisting of lowercase English letters.

Output

Display the sought number of triplets.

Example

standard input	standard output
abcababc	5

Note

In the example test case, the triplets are $(1, 3, 6)$, $(3, 4, 6)$, $(3, 4, 8)$, $(4, 5, 7)$, and $(5, 6, 8)$.

Problem D. Four Elements

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

You have a set A of non-negative integers that is a union of n ranges $[l_i; r_i]$. How many 4-element subsets with sum equal to s does A contain? Output this number modulo 998 244 353.

Input

The first line of the input contains two integers n and s ($1 \le n \le 400$; $0 \le s \le 8 \cdot 10^8$), denoting the number of ranges and the required sum.

Each of the following n lines contains two integers l_i and r_i ($0 \le l_i \le r_i \le 2 \cdot 10^8$; $r_i < l_{i+1}$), denoting the boundaries of the i-th range.

Output

Display the number of 4-element subsets of A with sum equal to s, modulo 998 244 353.

Example

standard input	standard output
2 17 1 3 5 8	4

Note

In the example test case, $A = \{1, 2, 3, 5, 6, 7, 8\}$. Its 4-element subsets with sum equal to 17 are $\{1, 2, 6, 8\}$, $\{1, 3, 5, 8\}$, $\{1, 3, 6, 7\}$, and $\{2, 3, 5, 7\}$.

Problem E. Five Points

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

You have placed n distinct points on a plane. Then you will draw a half-line starting from each point in a direction chosen uniformly at random and independently from other points. Find the probability that no two half-lines will intersect.

Input

The first line of the input contains a single integer n ($2 \le n \le 5$), denoting the number of points.

The i-th of the following n lines contains two integers x_i and y_i ($-100 \le x_i, y_i \le 100$), denoting the coordinates of the i-th point.

No two points coincide.

Output

Display the probability that no two half-lines will have a common point. Your answer must be correct to within an absolute or relative error of 10^{-9}.

Examples

standard input	standard output
2 0 0 1 1	0.7500000000000
3 7 8 7 7 8 8	0.4303385416667

Problem F. Six Words

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

The original title of this problem is "Line Line Graph Minimum Spanning Tree".

In this problem we will consider simple connected undirected graphs. Every edge e has *weight* $w(e)$. Every vertex v has *potential* $p(v)$.

The line graph of a graph G is another graph $L(G)$ that represents the adjacencies between edges of G. Formally, each vertex of $L(G)$ represents an edge of G, and two vertices of $L(G)$ are adjacent if and only if their corresponding edges share a common endpoint in G.

The potential of each vertex of $L(G)$ is equal to the weight of its corresponding edge in G. The weight of each edge e of $L(G)$ is equal to the potential of the vertex of G which is the common endpoint of the edges of G corresponding to the endpoints of e.

Note that whenever G is connected, $L(G)$ is connected as well.

The *minimum spanning tree* (MST) of a graph is a subset of its edges that connects all the vertices together, without any cycles and with the minimum possible total edge weight.

You are given a graph G with n vertices and m edges. The vertices are numbered from 1 to n, and the potential of vertex i of G is equal to i. The edges are numbered from 1 to m, and the weight of edge i of G is equal to i.

Find the total edge weight of the minimum spanning tree of $L(L(G))$.

Input

The first line of the input contains two integers n and m ($3 \le n \le 10^5$; $2 \le m \le \min(\frac{n(n-1)}{2}, 2 \cdot 10^5)$), denoting the number of vertices and the number of edges in G.

Each of the following m lines contains two integers u_i and v_i ($1 \le u_i, v_i \le n$; $u_i \ne v_i$), denoting the endpoints of the i-th edge.

There is at most one edge between any two vertices. The graph is connected.

Output

Display the total edge weight of the minimum spanning tree of the line graph of the line graph of G.

Examples

standard input	standard output
3 3 1 2 1 3 2 3	3
4 4 2 4 1 3 1 2 4 1	9

Note

In the first example test case, $L(L(G)) = G$, and the total edge weight of the MST of G is $1 + 2 = 3$.

Problem G. Seven Nevers

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

The *longest increasing subsequence* problem is to find a subsequence of a given sequence in which the subsequence's elements are in sorted order, lowest to highest, and in which the subsequence is as long as possible. This subsequence is not necessarily contiguous.

You are given a permutation of the first n positive integers $a_1, a_2, \ldots, a_n$ and an integer k. Your task is to find, for each i from 1 to $n - k + 1$, the length of the longest increasing subsequence of the sequence $a_1, a_2, \ldots, a_{i-1}, a_{i+k}, a_{i+k+1}, \ldots, a_n$ (in other words, the sequence obtained by erasing $a_i, a_{i+1}, \ldots, a_{i+k-1}$ from a).

Input

The first line of the input contains two integers n and k ($1 \le k < n \le 3 \cdot 10^5$), denoting the length of the given permutation and the number of consecutive elements to be removed.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le n$; $a_i \ne a_j$ for $i \ne j$), denoting the elements of the permutation.

Output

Display $n-k+1$ integers, one per line, where the i-th integer denotes the length of the longest increasing subsequence of the sequence $a_1, a_2, \ldots, a_{i-1}, a_{i+k}, a_{i+k+1}, \ldots, a_n$ for $i = 1, 2, \ldots, n - k + 1$.

Example

standard input	standard output
8 3 6 5 3 1 8 2 4 7	4 3 3 3 2 2

Problem H. Eight Sins

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

Does every interactive problem have to be binary search?

Somebody has secretly set n integers $a_1, a_2, \ldots, a_n$ such that $1 \le a_1 < a_2 < \ldots < a_n \le k$. You have to guess these integers one by one interacting with the testing system.

Initially $i = 1$. To make a guess, you may send an integer x such that $1 \le x \le k$. The system will reply with a string containing a single character:

- "`>`" if $a_i > x$;
- "`<`" if $a_i < x$;
- "`=`" if $a_i = x$.

In case $a_i = x$, the value of i is increased by 1. You win after you guess all n integers correctly, that is, when i reaches $n + 1$.

Note that the values of $a_1, a_2, \ldots, a_n$ are not chosen in advance by the interactor, but all replies to your queries will be consistent with some valid set of $a_1, a_2, \ldots, a_n$ at any time.

Interaction Protocol

First, the testing system writes a single line containing two integers n and k ($1 \le n \le 100$; $n \le k \le 10^9$), denoting the number of integers to guess and their maximum allowed value.

Your solution shall print requests. Each request consists of a single integer x on a single line. The testing system responds with a single line containing a single character "`>`", "`<`", or "`=`" as described in the problem statement.

Do not forget to flush the output after each request!

Your solution must terminate gracefully after receiving "`=`" from the testing system exactly n times.

Your solution is allowed to issue at most 2600 requests.

Example

standard input	standard output
3 6 < < = > = =	6 3 1 3 4 5

Problem I. Nine Judges

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

It is time to choose problems for the XCPC finals! There are k problems proposed for the contest, and exactly p of them must be chosen for the final problem set.

The problems have been evaluated by n judges. Each judge has built their own preference list containing all the proposed problems, in order from the problem they like the most to the problem they like the least.

There is still plenty of time until the XCPC finals... well, nobody knows if it is going to be held at all. Therefore, the actual problem set is going to be chosen by an infinite procedure.

Initially, $t = 0$ and S_0 contains p problems chosen from the proposed k problems uniformly at random. Then, the following sequence of actions will be repeated indefinitely:

- Choose problems i and j such that $i \in S_t$ and $j \notin S_t$ uniformly at random.
- If at least $\frac{n+1}{2}$ judges have put j before i in their preference list, let $S_{t+1} = S_t \setminus \{i\} \cup \{j\}$. Otherwise, let $S_{t+1} = S_t$.
- Increase t by one.

A problem set S is called *plausible* if there exists a positive probability threshold ε such that there are infinitely many moments t when the probability of this set being equal to S_t is more than ε. Here is an equivalent mathematical notation:

$$S \text{ is plausible} \iff \exists \varepsilon > 0 \; \forall T \geq 0 \; \exists t \geq T\colon \; P(S_t = S) > \varepsilon$$

You would like to save the judges some time and find any plausible problem set.

Input

The first line of the input contains three integers n, k, and p ($1 \leq n \leq 9$; n is odd; $1 \leq p < k \leq 50\,000$), denoting the number of judges, the number of problems proposed, and the number of problems to be chosen.

Each of the following n lines contains k integers $a_{i,1}, a_{i,2}, \ldots, a_{i,k}$ ($1 \leq a_{i,j} \leq k$; $a_{i,j} \neq a_{i,j'}$ for $j \neq j'$), denoting the preference list of the i-th judge: $a_{i,1}$ is the index of the problem they like the most, $a_{i,2}$ is the index of the second most liked problem, and so on.

Output

Display p distinct integers between 1 and k in any order, denoting any plausible set of problems.

Example

standard input	standard output
3 5 3 3 2 5 1 4 1 4 2 5 3 2 4 5 1 3	2 1 4

Note

In the example test case, some other possible answers are `1 2 5` and `2 4 5`.

Problem J. Ten Ranges

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

An integer x is called a *decimal subsequence* of an integer y if the decimal representation of x can be obtained from the decimal representation of y by erasing some digits (possibly none, but not all).

An integer x is called a prime if $x \geq 2$ and its only positive integer divisors are 1 and x.

An integer x is called *secondary* if there is no integer y such that y is a decimal subsequence of x and y is a prime. For example, 168 is secondary as none of 1, 6, 8, 16, 18, 68, 168 is a prime, while 169 is not secondary as 19 is a prime.

Find the number of secondary integers in each of the given ranges $[l; r]$.

Input

The first line of the input contains a single integer n ($1 \leq n \leq 10$), denoting the number of ranges.

Each of the following n lines contains two integers l and r ($1 \leq l \leq r \leq 10^{18}$), denoting the boundaries of the range.

Output

Display the number of secondary integers in each of the given ranges, in order of input.

Example

standard input	standard output
3 8 16 157 174 42 239	5 4 36

Note

In the example test case, secondary integers belonging to $[8; 16]$ are 8, 9, 10, 14, and 16, and secondary integers belonging to $[157; 174]$ are 160, 164, 166, and 168.

Problem K. Eleven Problems

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

You have prepared a very nice contest with n problems. This contest has already been used at two training camps. After each camp, you ran a quick poll asking the contestants which problem they liked the most. At the i-th camp, $a_{i,j}$ contestants voted for problem j.

You are going to build two pie charts based on this data, one for each training camp. Recall that a pie chart is a circular statistical graphic, which is divided into slices to illustrate numerical proportion. In a pie chart, the arc length of each slice (and consequently its central angle and area) is proportional to the quantity it represents.

Both pie charts must have the same radius and consist of exactly n slices, one for each problem, and each slice must be a single circular sector. The area of slice i must be proportional to the number of votes for problem i at the corresponding training camp. The rotation and the order of the slices can be arbitrary and different for the two pie charts.

You want the resulting pie charts to look as similar as possible. To have an objective criterion, you define the *similarity* of two pie charts as follows. Suppose that the slices corresponding to problem i are colored into color i. Place one pie chart over the other one so that their boundaries match. The similarity of the pie charts is the ratio of the area of the pie charts colored into the same color to the overall pie chart area. Note that the similarity of the pie charts is independent of their radius.

Find a way to draw the pie charts to maximize their similarity.

Input

The first line of the input contains an integer n ($2 \le n \le 11$), denoting the number of problems.

Each of the following n lines contains two integers $a_{1,j}$ and $a_{2,j}$ ($1 \le a_{i,j} \le 10^6$), denoting the number of votes for the j-th problem at the first and the second training camps.

Output

Display the maximum possible similarity of the pie charts. Your answer must be correct to within an absolute error of 10^{-9}.

Examples

standard input	standard output
2 3 2 2 2	0.9
4 3 7 21 2 4 5 2 6	0.333333333333333